

P1143R2 Adding the `constexpr` keyword

```
const char *g() { return "dynamic initialization"; }
constexpr const char *f(bool p) { return p ? "constant initializer" : g(); }

constexpr const char *c = f(true); // OK.
constexpr const char *d = f(false); // ill-formed
```

Introduction

We're all familiar with the Static Initialization Order Fiasco. Static storage duration variables with dynamic initializers cause hard-to-find bugs caused by the indeterminate order of dynamic initialization.

However, static variables with constant initializers avoid these pitfalls by causing the initialization to take place at compile time. These variables can be safely used during dynamic initialization across translation units.

Unfortunately the rules for constant initialization are complex and change dialect to dialect. This makes it non-obvious what form of initialization is taking place simply by inspecting the code. We need a way to enforce that constant initialization is truly taking place. It's important to be able to fail fast instead of silently falling back on dynamic initialization.

This paper proposes the `constexpr` keyword.

The keyword can be applied to variable declarations with static storage duration. It requires that the variable have a "constant initializer". For example:

```
// Compiled as C++14
struct T {
    constexpr T(int) {}
    ~T(); // non-trivial
};
constexpr T x = {42}; // Initialization OK. Doesn't check destructor.
constexpr T y = 42; // error: variable does not have a constant initializer
// copy initialization is not a constant expression on a non-literal type in C++14.
```

Previous Discussions

`constexpr` as an attribute.

in r0 of this paper `constexpr` was proposed as an attribute. When this idea was presented, there was general concencious that this feature would be better suited as a keyword. First, `constexpr` enforces correctness and if compilers were allowed to ignore it as they can attributes, it would allow "ill-formed" programs to compile. Second, there was some discussion about the behavior of `[[constexpr]]` being out-of-scope for attributes (I don't believe this to be the case).

Applying `constexpr` to declarations or just definitions?

`constexpr` communicates the intention of the programmer to both the compiler and other programmers. In order for the compiler to enforce the intent, the specifier need only be present on the initializing declaration, and not

any other declarations. However, to express the intent to other programmers there is value in allowing the `constinit` specifier to appear on non-defining declarations. For example:

```
// Foo.h
struct Foo {
    constinit static int x;
};
// Foo.cpp
int Foo::x = 42;
```

`constinit` can also be useful to compilers for non-initializing declarations of `thread_local` variables:

```
extern thread_local constinit x;
int f() { return x; }
```

Without `constinit`, runtime code must be executed to perform a check of a guard variable and conditionally initialize `x` each time it is used. (Other techniques exist, but this approach is common.) If the variable is known to have constant initialization, this can be avoided.

After EWG discussion, `constinit` is permitted on any declarations of a variable; if it is present on some declaration but not on the initializing declaration, the program is ill-formed, no diagnostic required.

Wording

Modify [dcl.spec] as described below.

5.11 Keywords [lex.key]

Add `constinit` to Table 5 (Keywords).

9.1 Specifiers [dcl.spec]

1. The specifiers that can be used in a declaration are:

decl-specifier:

...
`constinit`

2. Each `decl-specifier` shall appear at most once in a complete `decl-specifier-seq`, except that `long` may appear twice. ~~The `constexpr` and `constexpr` `decl-specifiers` shall not both appear in a `decl-specifier-seq`. At most one of the `constexpr`, `constexpr`, and `constexpr` keywords shall appear in a `decl-specifier-seq`.~~

9.1.X The `constinit` specifier [dcl.constinit]

Add the new subclause under [dcl.spec] with the following wording.

1. The `constinit` specifier shall be applied only to a declaration of a variable with static or thread storage duration. If the specifier is applied to any declaration of a variable, it shall be applied to the initializing declaration. No diagnostic is required if no `constinit` declaration is reachable at the point of the initializing declaration.
2. If a variable declared with the `constinit` specifier has dynamic initialization ([basic.start.dynamic]), the program is ill-formed. [Note: The `constinit` specifier ensures that the variable is initialized during static initialization ([basic.start.static]). --- end note]

3. [Example:

```
const char *g() { return "dynamic initialization"; }  
constexpr const char *f(bool p) { return p ? "constant initializer" : g(); }
```

```
constexpr const char *c = f(true); // OK.  
constexpr const char *d = f(false); // ill-formed
```

--- end example]

15.10 Predefined macro names [cpp.predefined]

Add a feature test macro `__cpp_constinit` with a suitable value to Table 17 (Feature-test macros).

C.5.1 Clause 5: lexical conventions [diff.cpp17.lex]

Add a bullet to the "New keywords" change in paragraph 2:

- The `constexpr` keyword is added to prevent unintended dynamic initialization.

References

- [Clang's \[\[require_constant_initialization\]\] documentation](#)
- [A Clang implementation of the r0 proposal](#)
- [Definition of *Constant Initialization* from \[basic.start.static\]](#)